

# 디자인 명세서

## OH-MY-NEURO Local RAG Document Search System

SWE3002 Software Engineering

OH-MY-NEURO Team

Document Date: 31 May 2026

|                                            |          |
|--------------------------------------------|----------|
| <b>1 Purpose</b>                           | <b>3</b> |
| 1.1 Readership                             | 3        |
| 1.2 Scope                                  | 3        |
| 1.3 Objective                              | 3        |
| 1.4 Document Structure                     | 4        |
| <b>2 Introduction</b>                      | <b>4</b> |
| 2.1 Objectives                             | 4        |
| 2.2 Applied Diagrams                       | 5        |
| 2.2.1 Used Tools                           | 5        |
| 2.2.2 Context Diagram                      | 5        |
| 2.2.3 Use Case Diagram                     | 5        |
| 2.2.4 Sequence Diagram                     | 5        |
| 2.2.5 Class Diagram and Data Model Diagram | 5        |
| 2.2.6 Entity Relationship Diagram          | 5        |
| 2.2.7 Project Scope                        | 6        |
| 2.2.8 References                           | 6        |
| 2.3 Key Design Characteristics             | 6        |
| <b>3 System Architecture - Overall</b>     | <b>6</b> |
| 3.1 Objectives                             | 6        |
| 3.2 System Organization                    | 6        |
| 3.2.1 System Diagram                       | 7        |
| 3.2.2 런타임 동작 요약                            | 8        |
| 3.3 Use Case Diagram                       | 8        |
| 3.4 Use Case 범위와 API 연결                    | 9        |
| <b>4 System Architecture - Frontend</b>    | <b>9</b> |
| 4.1 Objectives                             | 9        |
| 4.2 Frontend Components                    | 9        |
| 4.2.1 페이지별 상태와 이벤트 책임                      | 10       |
| 4.3 Frontend Data Flow                     | 10       |
| 4.4 Chat UI Sequence                       | 10       |
| 4.5 Chat 스트리밍 처리 책임                        | 11       |

|                                        |           |
|----------------------------------------|-----------|
| <b>5 System Architecture - Backend</b> | <b>11</b> |
| 5.1 Objectives                         | 11        |
| 5.2 API Layer                          | 12        |
| 5.3 Ingestion Subcomponents            | 12        |
| 5.4 RAG Subcomponents                  | 13        |
| 5.5 MCP Subcomponent                   | 13        |
| 5.6 CLI Subcomponent                   | 14        |
| 5.7 백엔드 클래스와 의존성 경계                    | 14        |
| <b>6 Protocol Design</b>               | <b>14</b> |
| 6.1 Objectives                         | 14        |
| 6.2 Health and Bootstrap               | 14        |
| 6.3 Chat Protocol                      | 15        |
| 6.4 Vault Protocol                     | 16        |
| 6.5 Settings Protocol                  | 17        |
| 6.6 Index Maintenance Protocol         | 17        |
| 6.7 Error Handling                     | 17        |
| 6.8 Endpoint Summary                   | 18        |
| 6.9 프로토콜 계약 보완                         | 19        |
| <b>7 Database Design</b>               | <b>19</b> |
| 7.1 Objectives                         | 19        |
| 7.2 ChromaDB Collection                | 20        |
| 7.2.1 청크 식별자와 메타데이터 규칙                 | 20        |
| 7.3 VaultState                         | 21        |
| 7.4 Browser Local Storage              | 21        |
| 7.5 Configuration Data                 | 21        |
| 7.5.1 AppConfig 기본값                    | 21        |
| 7.6 ER Diagram Interpretation          | 21        |
| 7.7 Relational Schema Compatibility    | 22        |
| 7.8 SQL DDL Applicability              | 22        |
| <b>8 Testing Plan</b>                  | <b>22</b> |
| 8.1 Objectives                         | 22        |
| 8.2 Testing Policy                     | 22        |
| 8.2.1 Development Testing              | 22        |
| 8.2.2 Release Testing                  | 22        |
| 8.2.3 User Testing                     | 23        |
| 8.2.4 Testing Cases                    | 23        |
| 8.3 테스트 범위 보완                          | 24        |
| <b>9 Development Plan</b>              | <b>24</b> |
| 9.1 Objectives                         | 24        |
| 9.2 Backend Environment                | 24        |
| 9.3 Frontend Environment               | 24        |

|                                          |           |
|------------------------------------------|-----------|
| 9.4 Configuration Environment.....       | 24        |
| 9.5 Constraints.....                     | 25        |
| 9.6 Assumptions and Dependencies.....    | 25        |
| 9.7 Implementation Roadmap.....          | 25        |
| 9.8 개발 품질 기준.....                        | 25        |
| <b>10 Supporting Information.....</b>    | <b>26</b> |
| 10.1. Software Design Specification..... | 26        |
| 10.2. Document History.....              | 26        |

# 1 Purpose

본 문서는 OH-MY-NEURO 시스템의 설계 명세서이다. OH-MY-NEURO는 사용자의 로컬 Vault 디렉토리에 있는 PDF, DOCX, TXT, Markdown 문서를 인덱싱하고, FastAPI/React 기반 UI와 CLI를 통해 출처가 포함된 답변을 제공하는 로컬 RAG 문서 검색 시스템이다.

## 1.1 Readership

이 문서는 다음 독자를 대상으로 한다. 첫째, OH-MY-NEURO의 프론트엔드, 백엔드, RAG, 저장소 계층을 구현하거나 유지보수하는 개발자이다. 둘째, 소프트웨어 공학 수업의 평가자와 조교이다. 셋째, 시스템의 기능 범위, 데이터 흐름, 테스트 계획을 검토해야 하는 프로젝트 이해관계자이다.

## 1.2 Scope

본 문서가 다루는 설계 범위는 다음과 같다. 포함 범위는 로컬 Vault 설정, 파일 스캔과 델타 동기화, 문서 로딩, 청킹, ChromaDB 벡터 저장, LangGraph 기반 Corrective RAG, OpenAI LLM 호출, 선택적 Korean-law MCP 라우팅, React 기반 Chat/Vault/Settings UI, CLI 명령이다. 제외 범위는 다중 사용자 인증, 클라우드 배포, 원본 문서 파일의 중앙 저장, 권한 기반 문서 공유, 실시간 협업 편집이다.

## 1.3 Objective

본 문서의 목표는 구현 코드와 일관성을 유지하는 시스템 설계 및 동작 기준을 제시하는 것이다. 특히 사용자가 로컬 문서를 Vault로 지정한 뒤 동기화하고, 질문을 입력하면 검색-평가-재작성-생성 흐름을 통해 출처가 붙은 답변을 받는 전체 동작을 설명한다. 또한 프론트엔드 컴포넌트, 백엔드 모듈, API 프로토콜, 데이터 저장 구조, 테스트 및 개발 계획을 명확히 정의한다. 모든 설계는 요구사항 명세서의 기능적, 비기능적 요구사항을 기반으로 작성되었다.

## 1.4 Document Structure

1장은 문서 목적과 범위를 설명한다.

2장은 설계에 사용된 다이어그램과 작성 기준을 소개한다.

3장은 전체 시스템 아키텍처와 사용 사례를 설명한다.

4장은 React 프론트엔드 구조를 설명한다.

5장은 FastAPI 백엔드, ingestion, RAG, storage, MCP 구조를 설명한다.

6장은 HTTP API와 스트리밍 프로토콜을 설명한다.

7장은 Database Design 관점에서 ChromaDB, VaultState, 브라우저 localStorage의 데이터 저장 구조와 relational schema 비적용 사유를 설명한다.

8장은 테스트 정책과 주요 테스트 케이스를 설명한다.

9장은 개발 환경, 제약 조건, 의존성을 설명한다.

10장은 참고 정보와 문서 변경 이력을 제공한다.

## 2 Introduction

OH-MY-NEURO는 로컬 문서를 대상으로 하는 RAG 애플리케이션이다. 사용자는 브라우저 UI에서 Vault 경로를 설정하고 SYNC를 실행한다. 백엔드는 문서를 읽어 청크로 나눈 뒤 OpenAI embedding을 사용해 ChromaDB에 저장한다. 이후 Chat 화면의 질문은 FastAPI의 NDJSON 스트리밍 API를 통해 RAGService로 전달되고, LangGraph 그래프가 검색, 관련성 평가, 질문 재작성, 선택적 MCP 검색, 답변 생성을 수행한다.

### 2.1 Objectives

이 장의 목적은 설계 문서에서 사용하는 다이어그램과 표기법을 정의하고, 프로젝트 범위를 코드베이스 기준으로 고정하는 것이다. 본 명세는 요구사항 명세서에서 정의된 요구사항을 충족하도록 설계되었으며, 설계와 구현의 일관성을 보장하기 위해 실제 모듈 구조 및 인터페이스와 교차 검증되었다.

### 2.2 Applied Diagrams

#### 2.2.1 Used Tools

다이어그램은 코드베이스 분석 결과를 바탕으로 직접 생성한 PNG 자료이다. 그림 생성에는 로컬 Python/Pillow 기반 도식화 스크립트를 사용했다. 다이어그램의 엔티티, 모듈명, API명은 app/ 및 frontend/src/ 경로의 실제 파일 구조와 함수명을 기준으로 했다.

## 2.2.2 Context Diagram

Context Diagram은 End User, React UI/CLI, FastAPI, OpenAI API, MCP server, local persistent stores 사이의 입력/출력 경계와 책임 분리를 보여준다.

## 2.2.3 Use Case Diagram

Use Case Diagram은 End User와 Developer/Operator가 수행하는 주요 행위를 표현한다. 일반 사용자는 Vault 경로 설정, 동기화, 문서 질문, 출처 포함 답변 확인, 설정 변경을 수행한다. 개발자 또는 운영자는 CLI 실행, MCP 서버 설정, 인덱스 초기화 등을 수행한다.

## 2.2.4 Sequence Diagram

Sequence Diagram은 ChatPage에서 질문이 입력된 뒤 FastAPI, RAGService, ChromaDB, OpenAI/MCP를 거쳐 NDJSON 스트림으로 답변이 렌더링되는 순서를 표현한다.

## 2.2.5 Class Diagram and Data Model Diagram

Class/Data Model Diagram은 Frontend page component, FastAPI router, RAGService, LangGraph node, ChromaStore, VaultState 같은 주요 class/module 경계를 설명하고, FileEntry, LoadedPage, Chunk, RetrievedChunk, ChatResponse, VaultState 등 런타임 객체의 변환 관계를 표현한다.

## 2.2.6 Entity Relationship Diagram

OH-MY-NEURO는 관계형 DB를 직접 사용하지 않으므로 ER Diagram은 전통적인 table 중심이 아니라 vector collection, VaultState, browser localStorage의 논리 엔티티 관계로 해석한다. Figure 7.1은 FileEntry, LoadedPage, Chunk, RetrievedChunk, ChatResponse, VaultState 사이의 변환 관계를 ERD 대체 자료로 제공한다.

## 2.2.7 Project Scope

프로젝트 범위는 로컬 문서 검색과 질의응답에 집중된다. 문서 수집은 로컬 디렉토리 기반이고, 검색 인덱스는 로컬 ChromaDB에 유지된다. LLM과 embedding은 OpenAI API에 의존한다. MCP는 법령 관련 질문을 보강하기 위한 선택적 외부 컨텍스트 경로로 설계되어 있다.

## 2.2.8 References

참조 자료는 현재 저장소의 README.md, pyproject.toml, configs/app.yaml, app/api, app/ingestion, app/rag, app/storage, app/vault, app/mcp, frontend/src 구현이다. .docs 내부의 디자인 명세서 예시와 디자인명세서 구조 PDF는 문서 구성과 장 흐름을 맞추기 위한 형식 참고 자료로 사용했다.

## 2.3 Key Design Characteristics

본 시스템의 설계는 다음 핵심 특성을 중심으로 결정되었다. 이 시스템은 사용자의 로컬 Vault를 원본 저장소로 유지하고, 백엔드는 파일을 읽어 검색 가능한 청크와 메타데이터만 ChromaDB에 저장한다. 원본 파일 중앙 저장, 사용자 계정, 클라우드 동기화는 이 시스템의 설계 범위에 포함되지 않는다. 핵심 흐름은 델타 동기화, Corrective RAG, 출처 기반 답변, NDJSON 스트리밍, 선택적 MCP 보강으로 요약된다. 델타 동기화는 `relative_path`, `mtime_ns`, `size`, `content_hash`를 비교해 변경 파일만 처리한다. Corrective RAG는 `retrieve` -> `grade` -> `rewrite` -> `route` -> `mcp` -> `prepare_context` -> `generate` 흐름을 사용하고, 검색 근거가 부족하면 한 번 재작성한 뒤 다시 검색한다.

## 3 System Architecture - Overall

### 3.1 Objectives

이 장은 OH-MY-NEURO의 전체 구성을 설명한다. 시스템은 로컬 입력, UI, API 서버, ingestion, RAG, 저장소, 외부 LLM/MCP의 결합으로 구성된다.

### 3.2 System Organization

전체 시스템은 세 가지 사용자 진입점을 가진다. 첫째, React SPA는 브라우저에서 Chat, Vault, Settings 화면을 제공한다. 둘째, FastAPI 서버는 `/api` 하위 엔드포인트와 `frontend/dist` 정적 파일을 제공한다. 셋째, `oh-my-neuro CLI`는 `ui`, `sync`, `vault`, `status`, `ask`, `list`, `clear` 명령을 제공한다.

FastAPI 계층은 `health`, `bootstrap`, `chat`, `vault`, `settings` 라우터로 분리되어 있다. Chat 요청은 RAGService로 위임된다. Vault 동기화 요청은 ingestion pipeline으로 전달된다. Settings 요청은 실행 중 설정 일부를 수정하고 RAGService, MCP client, vector store 캐시를 초기화한다.

RAG Core는 RAGService와 LangGraph 그래프로 구성된다. 그래프 노드는 `retrieve`, `grade`, `rewrite`, `route`, `mcp`, `prepare_context`, `generate` 순서로 동작한다. 검색 결과가 충분하지 않으면 질문을 재작성하고 재검색한다. 법령 키워드가 있고 MCP가 활성화되어 있으면 Korean-law MCP 도구를 호출한다.

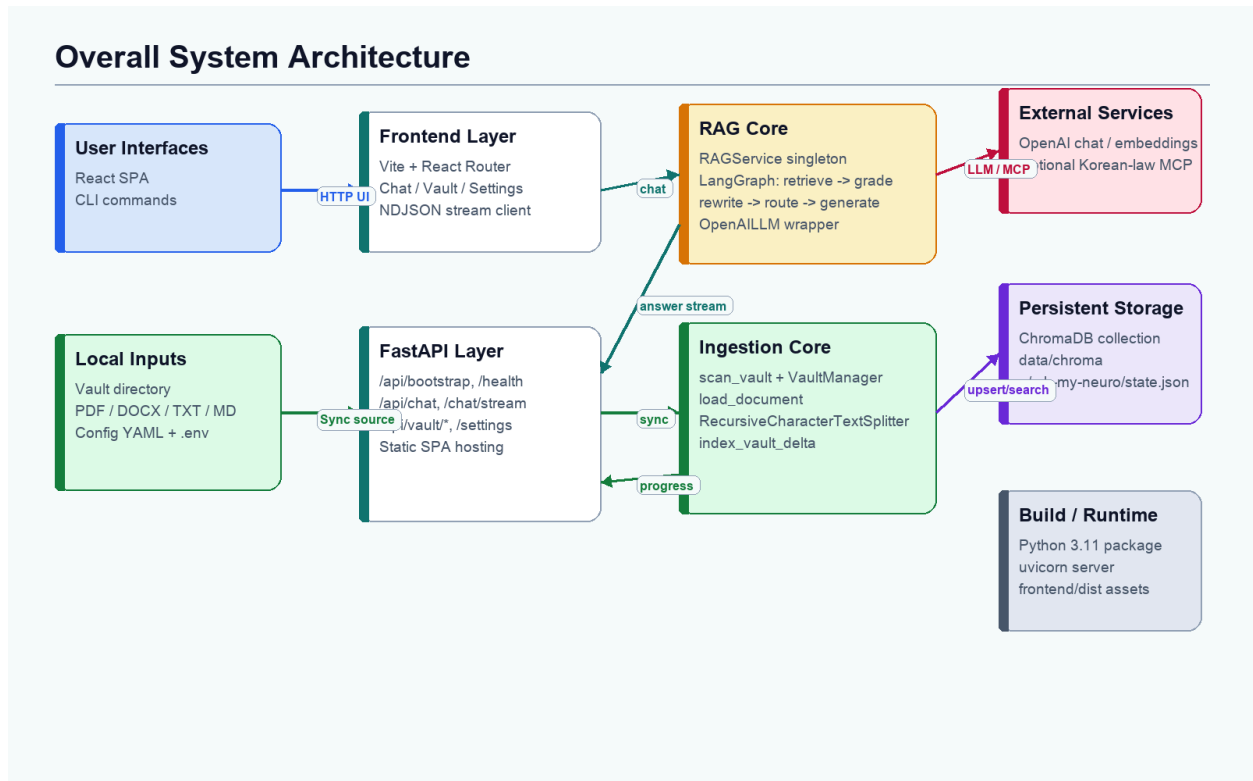
Ingestion Core는 `scan_vault`, `VaultManager.compute_delta`, `load_document`, `chunk_pages`, `ChromaStore.upsert_chunks`로 구성된다. 파일 변경 여부는 `relative_path`, `mtime_ns`, `size`, `content_hash`를 기준으로 판단한다.

#### 3.2.1 System Diagram

Figure 3.1은 전체 모듈 경계를 요약한다. React UI와 CLI는 사용자 진입점이고 FastAPI는 HTTP API와 정적 UI 서버를 담당한다. RAG Core는 검색, 평가, 재작성, 생성 책임을 가진다. Ingestion Core는 Vault

파일을 추출 가능한 텍스트와 청크로 변환한다. Persistent stores는 ChromaDB, VaultState JSON, browser localStorage로 분리된다.

Figure 3.1 Overall System Architecture



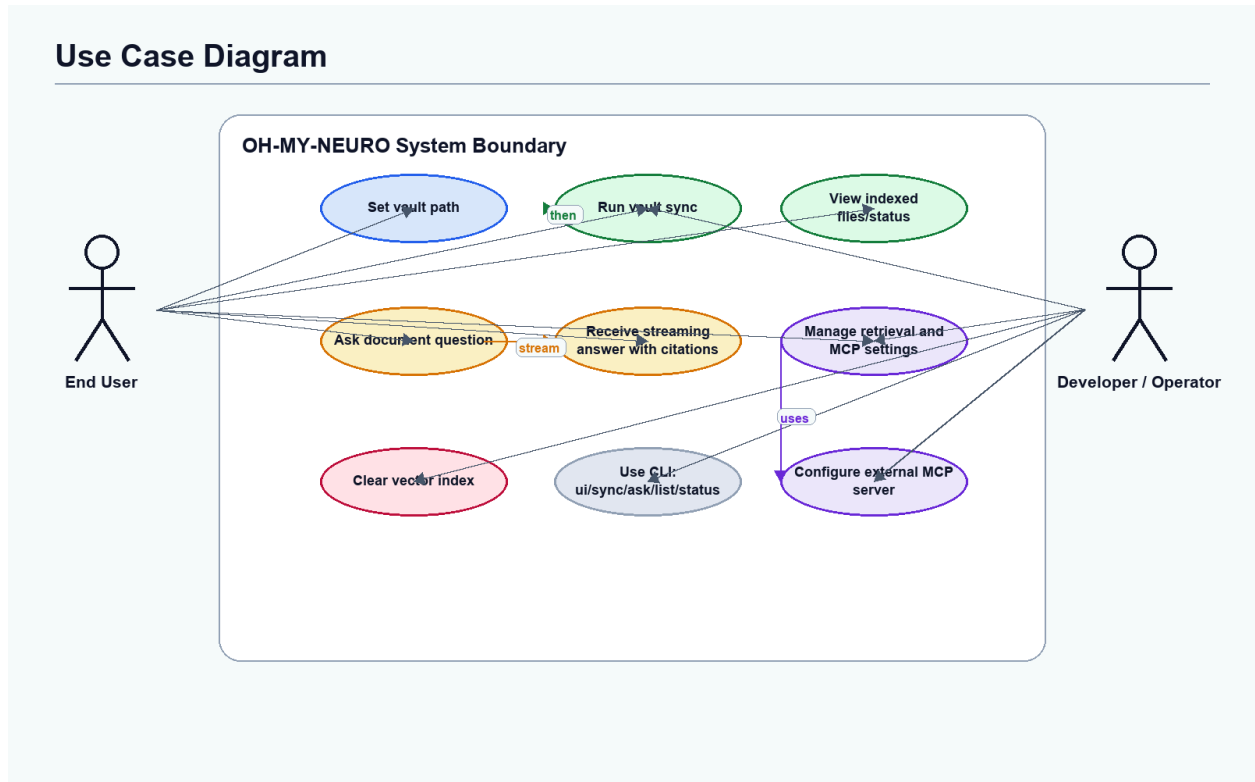
### 3.2.2 런타임 동작 요약

사용자가 Vault 경로를 설정하면 VaultState가 ~/.oh-my-neuro/state.json에 저장되고 onboarding\_completed가 true가 된다. SYNC 요청은 FastAPI의 /api/vault/sync에서 별도 스레드로 ingestion pipeline을 실행하며, progress/done/error 이벤트를 NDJSON으로 UI에 전달한다. 질문 요청은 /api/chat 또는 /api/chat/stream으로 들어오며 RAGService가 OpenAI API key, ChromaDB 인덱스 상태, 일반 대화 여부를 먼저 확인한다. 일반 대화는 RAG를 우회하고, 검색 질문은 LangGraph 경로로 들어가 ChromaDB 검색 결과와 선택적 MCP 결과를 Citation으로 합쳐 답변에 연결한다.

## 3.3 Use Case Diagram

주요 사용 사례는 다음과 같다. 사용자는 Vault 경로를 설정한 뒤 첫 SYNC를 실행한다. 이후 Vault 화면에서 파일 목록과 마지막 동기화 시간을 확인한다. Chat 화면에서는 질문을 입력하고 출처 badge가 포함된 답변을 받는다. Settings 화면에서는 top\_k, fetch\_k, chunk\_size, chunk\_overlap, max\_rewrites, relevance\_threshold와 MCP 활성화 여부를 조정한다. 운영자는 CLI로 동일한 기능 일부를 수행하거나 인덱스를 초기화한다.

Figure 3.2 Use Case Diagram



### 3.4 Use Case 범위와 API 연결

Vault 관리 범위는 Vault 경로 설정, 동기화, 인덱싱 파일 조회, 인덱스 초기화이다. 각각 POST /api/vault, POST /api/vault/sync, GET /api/vault/files, POST /api/index/clear와 연결된다.

질의응답 범위는 동기 응답, 스트리밍 응답, 이전 대화 history를 포함한 질문 처리이다. POST /api/chat은 완성 응답을 반환하고 POST /api/chat/stream은 meta, token, error, done 이벤트를 순차 전송한다.

설정 범위는 retrieval, mcp, ui 섹션 일부 변경이다. PATCH /api/settings는 알려진 key만 허용하고, 변경 후 RAGService 및 필요한 캐시를 초기화해 다음 요청부터 새 값을 사용하게 한다.

## 4 System Architecture - Frontend

### 4.1 Objectives

프론트엔드는 사용자가 로컬 문서 검색 시스템을 반복적으로 사용할 수 있도록 경량 콘솔 UI를 제공한다. 핵심 요구사항은 Vault onboarding, 동기화 진행률 표시, 스트리밍 답변 렌더링, 출처 표시, 설정 변경, 다크 모드, 최근 대화 유지이다.



## 4.2 Frontend Components

App.tsx는 bootstrap API를 호출해 초기 상태를 확인하고, onboarding 여부에 따라 /onboarding 또는 /chat으로 라우팅한다. AppShell.tsx는 좌측 네비게이션, 최근 대화 목록, Vault 상태 badge, 다크 모드 버튼을 제공한다.

OnboardingPage.tsx는 Vault path 입력과 첫 SYNC 실행을 담당한다. setVaultPath 이후 streamSync를 호출하며, progress/done/error 이벤트를 Progress 컴포넌트로 표시한다.

VaultPage.tsx는 Vault 상태, 인덱싱 파일 목록, 검색 필터, SYNC 실행, 인덱스 초기화를 담당한다. 파일 목록은 /api/vault/files 응답을 기반으로 렌더링되며, 크기와 동기화 시각을 사용자 친화적으로 표시한다.

ChatPage.tsx는 질문 입력, NDJSON 스트리밍 수신, ReactMarkdown 렌더링, citation badge 표시, stop 버튼, 세션 저장을 담당한다. 세션은 useSessions 혹은 통해 localStorage에 저장된다.

SettingsPage.tsx는 retrieval 설정과 MCP 토글을 수정한다. patchSettings 호출 후 서버 측 RAGService와 MCP client가 reset되어 다음 요청부터 새 설정이 적용된다.

### 4.2.1 페이지별 상태와 이벤트 책임

ChatPage는 messages, inputText, isStreaming, citations 상태를 중심으로 질문 제출, meta/token/error/done 체크 처리, stop 버튼, Markdown 렌더링을 담당한다. useSessions는 대화 index와 세션 본문을 localStorage key로 분리해 저장한다.

VaultPage는 vaultPath, syncProgress, indexedFiles, isSyncing 상태를 사용해 Vault 설정, 파일 목록 조회, 동기화 스트림 수신, clear index 액션을 연결한다. 동기화 진행률은 fraction이 있는 progress 이벤트에만 수치형 progress bar로 반영한다.

SettingsPage는 retrieval 값과 MCP 토글을 편집한다. 저장 후 서버 응답을 기준으로 화면 상태를 갱신하므로, 프론트엔드는 임의의 로컬 설정을 authoritative state로 간주하지 않는다.

## 4.3 Frontend Data Flow

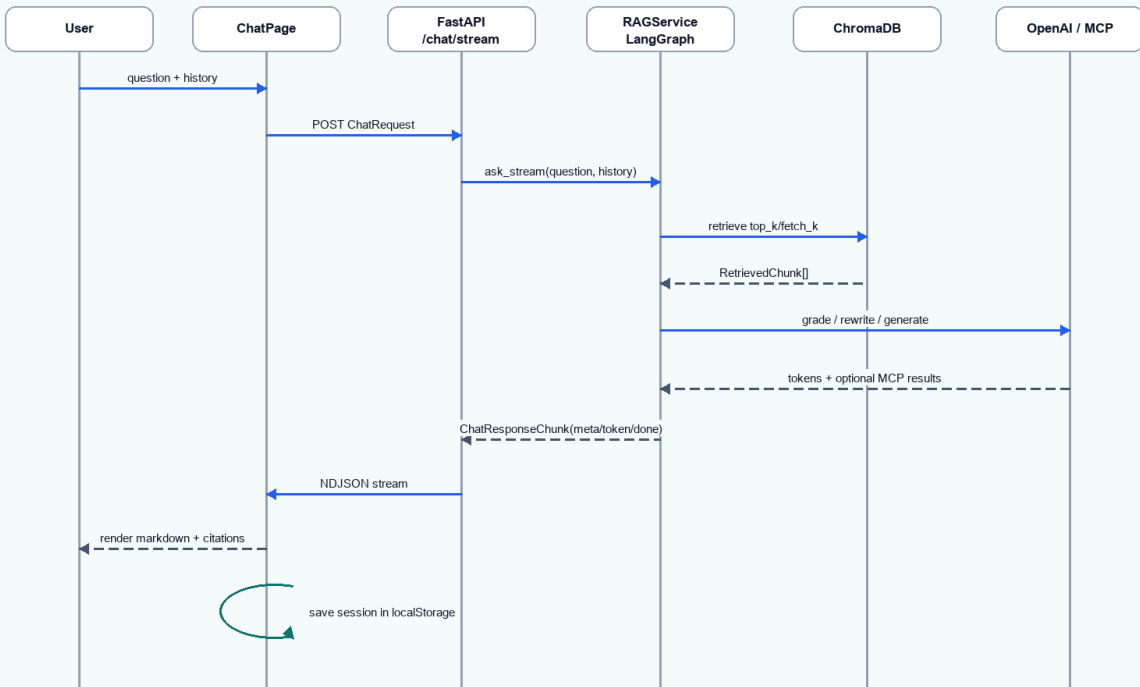
프론트엔드는 frontend/src/lib/api.ts에 정의된 fetch wrapper를 통해 서버와 통신한다. 일반 JSON 응답은 request 함수가 처리하고, Chat 및 Sync 스트림은 requestStream 함수가 ReadableStream을 줄 단위로 파싱한다. 스트림 포맷은 application/x-ndjson이며 각 줄은 JSON 객체 하나이다.

## 4.4 Chat UI Sequence

다음 그림은 ChatPage에서 사용자가 질문을 보낸 뒤 답변이 스트리밍으로 표시되는 순서를 나타낸다.

**Figure 4.1 Chat Streaming Sequence**

## Chat Streaming Sequence



### 4.5 Chat 스트리밍 처리 책임

스트리밍 UI는 첫 meta 청크에서 citations, used\_mcp, rewritten\_question, retrieval\_count를 고정하고, 이후 token 청크를 현재 assistant message에 누적한다. error 청크는 비치명적 오류로 표시하고, done 청크에서 입력 잠금과 stop 상태를 해제한다.

이 설계는 RAG 컨텍스트 준비와 토큰 생성의 책임을 분리한다. RAGService.ask\_stream은 terminal="prepare\_context" 그래프로 출처와 컨텍스트를 먼저 만든 뒤, OpenAILLM.generate\_stream의 결과만 token 이벤트로 내려보낸다.

## 5 System Architecture - Backend

### 5.1 Objectives

백엔드는 UI와 CLI의 요청을 실제 문서 인덱싱, 검색, 답변 생성으로 연결한다. 백엔드는 FastAPI 서버, 설정 로더, Vault scanner, ingestion pipeline, ChromaDB 저장소, LangGraph RAG, MCP client, CLI entry point로 구성된다.

## 5.2 API Layer

`app/api/main.py`는 FastAPI 앱을 생성하고 CORS, 라우터, 정적 파일 서비스를 설정한다. `/api/health`는 앱 이름과 API key 설정 여부를 반환한다. `/api/bootstrap`은 UI 제목, 설명, onboarding 상태, Vault path, `indexed_files`, `last_sync_at`을 반환한다. `/api/chat`과 `/api/chat/stream`은 RAGService를 호출한다. `/api/vault`는 Vault path 설정, 상태 조회, 파일 목록, 동기화 스트림을 담당한다. `/api/settings`는 실행 중 설정 조회와 patch를 담당한다.

## 5.3 Ingestion Subcomponents

Vault scanner는 recursive 옵션, `include_extensions`, `max_file_mb`, `excluded_dirs`에 따라 파일을 필터링한다. 숨김 디렉토리나 제외 디렉토리는 `traversal`에서 제거한다. 지원 형식은 `.pdf`, `.docx`, `.txt`, `.md`, `.markdown`이다.

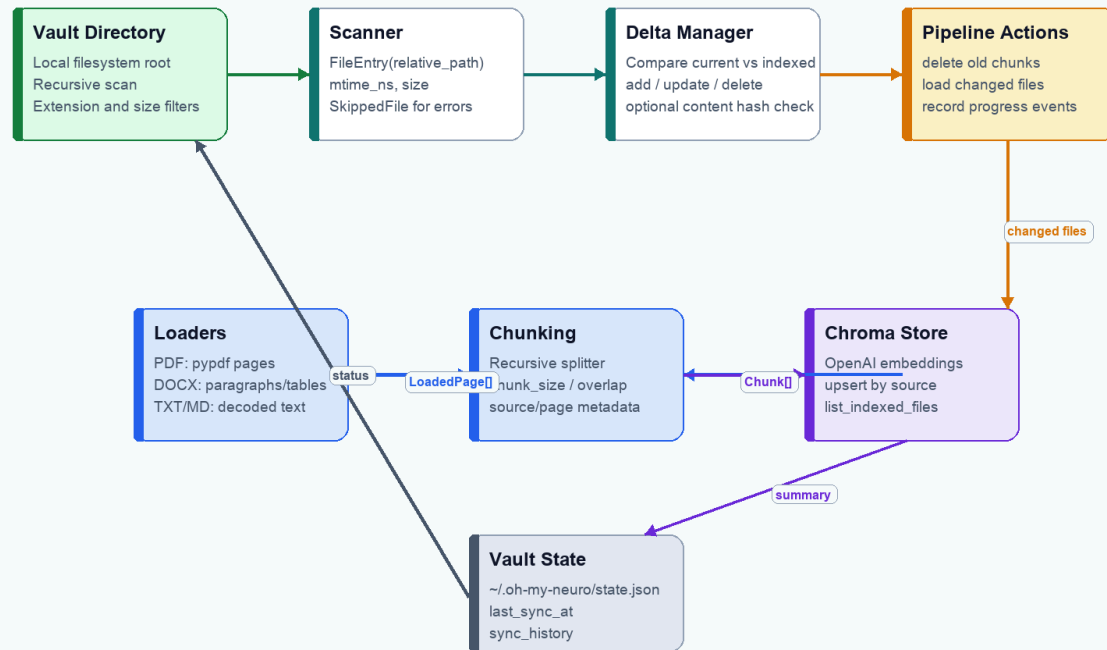
Document loaders는 PDF는 `pypdf`, DOCX는 `python-docx`, TXT/MD는 여러 인코딩 `utf-8`, `utf-8-sig`, `cp949`, `euc-kr` 순서로 읽는다. `LoadedPage` 객체는 `source`, `doc_type`, `page`, `location`, `text`를 가진다.

Chunking은 `RecursiveCharacterTextSplitter`를 사용한다. 기본 `chunk_size`는 1000, `chunk_overlap`은 150이다. 각 Chunk는 `chunk_id`, `content`, `source`, `page`, `location`, `doc_type`, `uploaded_at`, `extras`를 포함한다.

ChromaStore는 `chromadb.PersistentClient`를 사용해 `data/chroma` 경로의 `oh_my_neuro` collection을 관리한다. `embedding_function`은 `OpenAIEmbeddingFunction`이며 `embedding model`은 `text-embedding-3-large`로 설정되어 있다.

**Figure 5.1 Vault Synchronization Flow**

## Vault Synchronization Flow



## 5.4 RAG Subcomponents

RAGService는 UI/CLI가 호출하는 단일 진입점이다. ask는 완성 응답을 반환하고 ask\_stream은 meta, token, error, done 청크를 순서대로 yield한다. API key가 없거나 인덱스가 비어 있으면 사용자에게 즉시 안내 메시지를 반환한다.

OpenAILLM은 grade, rewrite, generate, classify\_intent, chat, chat\_stream, generate\_stream 기능을 제공한다. 일반 대화로 분류된 입력은 RAG 그래프를 우회한다. 검색 질문은 LangGraph로 전달된다.

LangGraph 파이프라인은 retrieve -> grade -> rewrite 또는 route -> optional mcp -> prepare\_context -> generate 순서로 실행된다. 검색 문서가 충분하지 않으면 max\_rewrites 범위 내에서 질문을 재작성한다. MCP 라우팅은 mcp.enabled와 law\_keywords 조건을 모두 만족해야 실행된다.

## 5.5 MCP Subcomponent

MCPClient는 configs/mcp\_servers.yaml을 로드하고 langchain-mcp-adapters의 MultiServerMCPClient를 초기화한다. 기본 서버는 korean\_law이며 stdio transport와 uvx mcp-server-korean-law 명령을 사용한다. MCP 결과는 MCPResult로 정규화되어 일반 Citation과 같은 방식으로 답변 컨텍스트에 합쳐진다.

## 5.6 CLI Subcomponent

app/main.py는 oh-my-neuro console script의 entry point이다. 기본 명령은 ui이다. sync는 Vault delta 동기화를 실행하고 진행률을 출력한다. vault는 Vault 경로를 설정한다. status는 Vault path, last\_sync\_at, indexed files 수를 출력한다. ask는 RAGService.ask 또는 ask\_stream을 호출한다. list는 인덱싱된 파일을 출력하고 clear는 --force 확인 후 ChromaDB collection을 초기화한다.

## 5.7 백엔드 클래스와 의존성 경계

RAGService는 UI와 CLI가 호출하는 단일 facade이다. 내부적으로 ChromaStore, OpenAILLM, MCPClient, LangGraph compiled graph를 lazy initialization하며, ask는 완성형 ChatResponse를 반환하고 ask\_stream은 ChatResponseChunk iterator를 반환한다.

OpenAILLM은 ChatOpenAI wrapper로 grade, rewrite, generate, classify\_intent, chat, chat\_stream, generate\_stream을 제공한다. 일시적 OpenAI API 장애는 tenacity로 최대 3회 재시도하고, 일반 대화로 분류된 입력은 문서 검색 없이 chat/chat\_stream 경로로 처리한다.

GraphDeps는 Retriever, LLMClient, MCPBridge Protocol을 통해 그래프가 구체 클래스에 직접 결합되지 않도록 한다. 현재 구현에서 ChromaStore는 Retriever, OpenAILLM은 LLMClient, MCPClient는 MCPBridge 역할을 수행한다.

Ingestion pipeline은 scan\_vault -> compute\_delta -> load\_document -> chunk\_pages -> upsert\_chunks 순서로 실행된다. 삭제는 먼저 delete\_by\_source로 반영하고, 수정/추가는 파일 로딩과 청킹 성공 후 ChromaDB에 upsert한다.

MCPClient는 lazy initialization과 실패 격리를 사용한다. 설정상 MCP가 꺼져 있거나 서버 초기화가 실패하면 내부 문서 검색 경로는 계속 동작하고, 법령 키워드와 서버 가용성이 모두 충족될 때만 search\_law 결과를 컨텍스트에 합친다.

# 6 Protocol Design

## 6.1 Objectives

프로토콜 설계의 목표는 브라우저 UI와 FastAPI 백엔드 사이의 요청/응답 형식을 명확히 정의하는 것이다. 모든 API는 /api prefix를 사용한다. 일반 응답은 JSON이며, Chat 및 Vault sync 진행률은 NDJSON 스트림이다.

## 6.2 Health and Bootstrap

GET /api/health의 응답 필드와 의미는 아래 표에 정리했다.

GET /api/bootstrap은 BootstrapResponse를 반환한다. 응답 필드와 의미는 아래 표에 정리했다. 프론트엔드는 이 응답으로 onboarding redirect와 AppShell badge를 결정한다.

**Figure 6.1 HealthResponse**

| 필드                 | 타입      | 설명                   |
|--------------------|---------|----------------------|
| ok                 | boolean | 항상 true              |
| app                | string  | 앱 이름                 |
| api_key_configured | boolean | OPENAI_API_KEY 설정 여부 |

**Figure 6.2 BootstrapResponse**

| 필드                   | 타입            | 설명                    |
|----------------------|---------------|-----------------------|
| title                | string        | UI 제목                 |
| description          | string        | UI 부제목                |
| onboarding_completed | boolean       | 온보딩 완료 여부             |
| vault_path           | string        | 현재 Vault 경로           |
| api_key_configured   | boolean       | API 키 설정 여부           |
| indexed_files        | integer       | 인덱싱된 파일 수             |
| last_sync_at         | string / null | 마지막 동기화 시각 (ISO 8601) |

## 6.3 Chat Protocol

POST /api/chat은 ChatRequest를 요청 본문으로 받고 ChatResponseDTO를 반환한다. 요청 및 응답 필드는 아래 표에 각각 정리했다.

POST /api/chat/stream은 같은 ChatRequest를 받지만 application/x-ndjson 형식의 ChatResponseChunk 스트림을 반환한다. 각 kind별 포함 필드와 의미는 아래 표에 정리했다.

**Figure 6.3 Post /api/chat - Request**

| 필드       | 타입            | 설명       |
|----------|---------------|----------|
| question | string        | 사용자 질문   |
| history  | ChatMessage[] | 이전 대화 이력 |

Figure 6.4 Post /api/chat - Response

| 필드                 | 타입            | 설명              |
|--------------------|---------------|-----------------|
| answer             | string        | LLM 생성 답변       |
| citations          | Citation[]    | 출처 목록           |
| used_mcp           | boolean       | MCP 법령 검색 사용 여부 |
| rewritten_question | string / null | 검색어 재작성 결과      |
| retrieval_count    | integer       | 사용된 문서 청크 수     |

Figure 6.5 Post /api/chat/stream

| kind  | 포함 필드                                                    | 설명                                        |
|-------|----------------------------------------------------------|-------------------------------------------|
| meta  | citations, used_mcp, rewritten_question, retrieval_count | 출처 및 메타정보. 첫 번째 이벤트로 전송.                  |
| token | text                                                     | 답변 텍스트 delta                              |
| error | text                                                     | 비치명적 오류 메시지                               |
| done  | text?                                                    | Stream 종료. Fallback 응답에서는 text에 완성 답변 포함. |

6.4 Vault Protocol

GET /api/vault/status는 VaultStatusResponse를 반환한다. POST /api/vault는 VaultPathRequest를 받아 경로를 검증한 뒤 VaultState를 저장한다. GET /api/vault/files는 IndexedFilesResponse를 반환한다. POST /api/vault/sync는 application/x-ndjson 스트림을 반환하며 progress, done, error 이벤트를 사용한다.

progress 이벤트는 kind, file, stage, fraction을 가진다. done 이벤트는 result와 skipped 목록을 가진다. error 이벤트는 text를 가진다. UI는 fraction 값이 숫자인 경우 Progress bar를 갱신하고, done 이벤트에서 100%로 표시한다.

## 6.5 Settings Protocol

GET /api/settings는 SettingsResponse를 반환하며, PATCH /api/settings는 SettingsPatch로 실행 중 설정 일부를 반영한다. 변경 가능한 섹션과 부작용은 아래 표에 정리했으며, 알 수 없는 key는 HTTP 400으로 거부된다.

**Figure 6.6 PATCH /api/settings**

| 필드        | 변경 가능한 하위 키                                                                  | 부작용                                     |
|-----------|------------------------------------------------------------------------------|-----------------------------------------|
| retrieval | top_k, fetch_k, chunk_size, chunk_overlap, max_rewrites, relevance_threshold | reset_service() 호출                      |
| mcp       | enabled, law_keywords                                                        | reset_mcp_client() → reset_service() 호출 |
| ui        | title, description, dark                                                     | reset_service() 호출                      |

## 6.6 Index Maintenance Protocol

POST /api/index/clear는 ClearRequest를 받는다. force가 true가 아니면 HTTP 400을 반환한다. force=true이면 clear\_all을 호출하고 reset\_vector\_store, reset\_service를 수행한 뒤 deleted\_chunks를 반환한다.

## 6.7 Error Handling

서버의 주요 오류 원인별 처리 방식과 HTTP 코드는 아래 표에 정리했다. 프론트엔드 request wrapper는 detail 또는 message를 추출해 사용자에게 표시하고, RAGService 내부 오류는 가능한 한 ChatResponse의 answer 또는 stream error chunk로 변환된다.

**Figure 6.7 ErrorHandling**

| 오류 원인                   | 처리 방식         | HTTP 코드 |
|-------------------------|---------------|---------|
| 존재하지 않는 Vault path      | HTTPException | 400     |
| force 없는 index clear 요청 | HTTPException | 400     |



|                     |                           |                   |
|---------------------|---------------------------|-------------------|
| 알 수 없는 settings key | HTTPException             | 400               |
| Pydantic 유효성 검사 실패  | FastAPI 자동 처리             | 422               |
| LLM 초기화 실패, 타임아웃    | answer 또는 error chunk로 변환 | 200 (stream 내 처리) |
| 스트리밍 생성 오류          | error chunk 전송 후 스트림 계속   | 200 (stream 내 처리) |

## 6.8 Endpoint Summary

주요 엔드포인트의 method, path, request, response 계약은 아래 표에 정리했다.

**Figure 6.8 Endpoint**

| Method | Path              | Request                        | Response                                                                                                                 |
|--------|-------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| GET    | /api/health       | 없음                             | HealthResponse(ok, app, api_key_configured)                                                                              |
| GET    | /api/bootstrap    | 없음                             | BootstrapResponse(title, description, onboarding_completed, vault_path, api_key_configured, indexed_files, last_sync_at) |
| POST   | /api/chat         | ChatRequest(question, history) | ChatResponseDTO(answer, citations, used_mcp, rewritten_question, retrieval_count)                                        |
| POST   | /api/chat/stream  | ChatRequest                    | application/x-ndjson<br>ChatResponseChunk stream                                                                         |
| POST   | /api/vault        | VaultPathRequest(path)         | VaultStatusResponse                                                                                                      |
| GET    | /api/vault/status | 없음                             | VaultStatusResponse                                                                                                      |
| GET    | /api/vault/files  | 없음                             | IndexedFilesResponse(files)                                                                                              |
| POST   | /api/vault/sync   | 없음                             | progress/done/error NDJSON stream                                                                                        |
| GET    | /api/settings     | 없음                             | SettingsResponse                                                                                                         |
| PATCH  | /api/settings     | SettingsPatch                  | SettingsResponse                                                                                                         |

|      |                  |                     |                               |
|------|------------------|---------------------|-------------------------------|
| POST | /api/index/clear | ClearRequest(force) | ClearResponse(deleted_chunks) |
|------|------------------|---------------------|-------------------------------|

## 6.9 프로토콜 계약 보완

ChatRequest.history는 user, assistant, system role을 갖는 ChatMessage 배열이며, 서비스 계층은 dict, tuple, ChatMessage 입력을 안전하게 정규화한다. ChatResponseDTO의 citations는 Citation 모델이며 source, location, page, doc\_type, kind, snippet으로 구성된다.

NDJSON 스트림은 줄 단위 JSON 객체를 전제로 한다. meta는 가능한 한 첫 번째 이벤트로 보내고, token은 텍스트 delta만 포함하며, done은 종료 신호이다. API key 없음, 빈 질문, 빈 Vault 같은 폴백 경로에서는 token 없이 done.text에 안내 문구가 들어갈 수 있다.

오류 처리는 HTTP 레벨과 응답 레벨로 나뉜다. 잘못된 Vault path, force 없는 index clear, 알 수 없는 settings key는 HTTP 400으로 반환한다. RAG 실행 중 LLM 초기화 실패, 타임아웃, 스트리밍 생성 오류는 사용자에게 설명 가능한 answer 또는 error chunk로 변환한다.

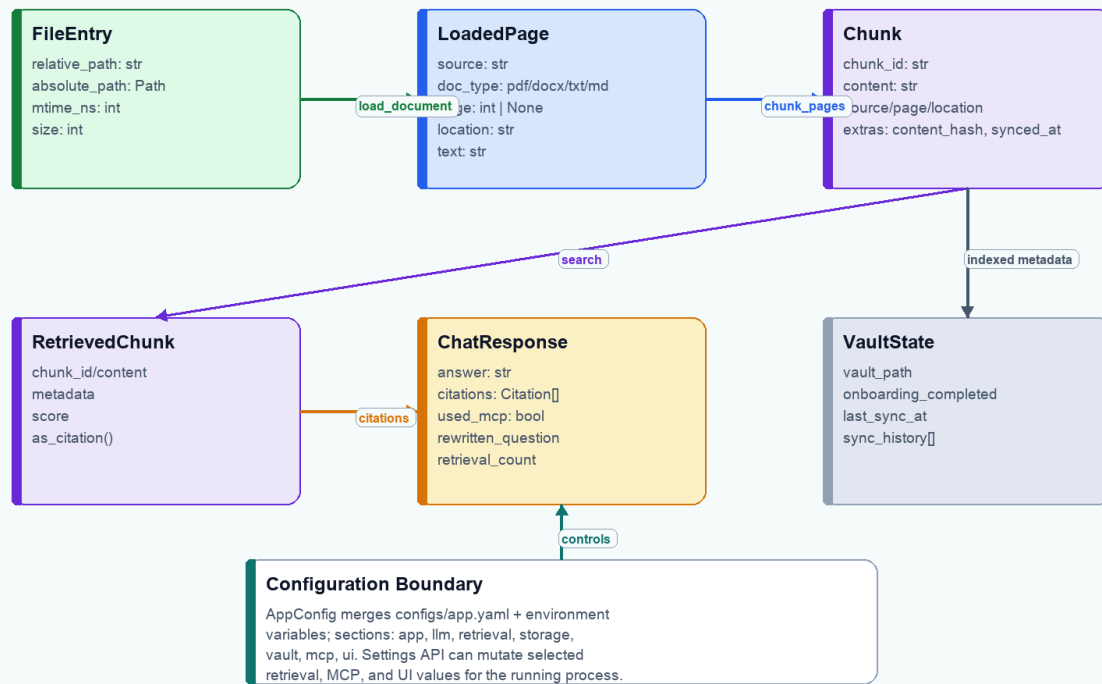
# 7 Database Design

## 7.1 Objectives

OH-MY-NEURO는 전통적인 관계형 데이터베이스를 사용하지 않는다. 영속 데이터는 ChromaDB vector collection, VaultState JSON 파일, 브라우저 localStorage에 분산 저장된다. 이 장은 Database Design 장의 역할에 맞추어 각 저장소의 책임, 논리 엔티티, relational schema 비적용 사유, SQL DDL 비적용 사유를 설명한다.

**Figure 7.1 Data Model / Runtime Objects**

## Data Model / Runtime Objects



## 7.2 ChromaDB Collection

ChromaDB collection 이름은 configs/app.yaml의 storage.collection\_name으로 정의되며 기본값은 oh\_my\_neuro이다. persist path는 data/chroma이다. 각 벡터 document는 Chunk.content이고 metadata는 source, page, location, doc\_type, uploaded\_at, relative\_path, mtime\_ns, size, content\_hash, synced\_at 등을 포함한다.

source는 청크의 논리적 출처이고 relative\_path는 Vault root 기준 경로이다. delete\_by\_source는 source 기준으로 기존 청크를 삭제한 뒤 upsert를 수행한다. list\_indexed\_files는 metadata를 집계해 파일별 상태를 만든다.

### 7.2.1 청크 식별자와 메타데이터 규칙

chunk\_id는 relative\_path와 청크 순번을 결합한 문자열이며, 동일 파일 재동기화 시 같은 source의 기존 청크를 삭제한 뒤 새 청크를 upsert한다. 이를 통해 파일 수정, 삭제, 재추가가 검색 결과에 중복으로 남지 않도록 한다.

metadata는 검색 결과 표시와 델타 판단을 동시에 지원한다. source, page, location, doc\_type은 Citation 렌더링에 사용되고, relative\_path, mtime\_ns, size, content\_hash, synced\_at은 list\_indexed\_files와 compute\_delta의 기준 데이터로 사용된다.

## 7.3 VaultState

VaultState는 ~/.oh-my-neuro/state.json에 저장된다. 필드는 vault\_path, onboarding\_completed, last\_sync\_at, sync\_history이다. sync\_history는 최대 20개로 제한된다. 저장은 tempfile과 os.replace를 사용한 atomic write 방식이다.

## 7.4 Browser Local Storage

프론트엔드는 최근 대화 세션을 브라우저 localStorage에 저장한다. 세션 index key는 omn-react-sessions이고, 개별 세션 key는 omn-react-session-{id} 형식이다. 각 세션은 ChatMessage 배열이다. 이 데이터는 서버로 저장되지 않으며 브라우저 로컬 상태이다.

## 7.5 Configuration Data

AppConfig는 configs/app.yaml과 환경 변수 OPENAI\_API\_KEY, OMN\_LOG\_LEVEL을 병합한다. 주요 섹션은 app, llm, retrieval, storage, vault, mcp, ui이다. MCP 서버 상세 설정은 configs/mcp\_servers.yaml에서 로드된다.

### 7.5.1 AppConfig 기본값

configs/app.yaml은 app, llm, retrieval, storage, vault, mcp, ui 섹션으로 나뉜다. 기본 chat/grader/rewriter model은 gpt-5.4-mini이고 embedding model은 text-embedding-3-large이다. retrieval 기본값은 top\_k=5, fetch\_k=12, chunk\_size=1000, chunk\_overlap=150, max\_rewrites=1, relevance\_threshold=0.5이다. vault는 .pdf, .docx, .txt, .md, .markdown을 포함하고 단일 파일 최대 크기는 50MB로 제한한다.

MCP는 configs/app.yaml에서 기본적으로 disabled이며, 활성화된 경우 configs/mcp\_servers.yaml의 korean\_law stdio 서버와 law\_keywords를 사용한다. ui 기본 host는 127.0.0.1, port는 7860이다.

## 7.6 ER Diagram Interpretation

Figure 7.1은 전통적 ERD 대신 구현 객체와 저장소 사이의 관계를 보여준다. FileEntry는 Vault scanner의 파일 단위 입력이고, LoadedPage는 loader가 추출한 페이지/문단 단위 텍스트이다. Chunk는 ChromaDB에 저장되는 검색 단위이며 RetrievedChunk는 검색 점수와 citation 생성을 위한 런타임 객체이다.

## 7.7 Relational Schema Compatibility

관계형 schema는 현재 구현 대상이 아니다. 다만 설계 관점에서 ChromaDB collection은 chunks 테이블, VaultState JSON은 vault\_state 테이블, browser localStorage는 chat\_sessions 테이블에 대응되는 논리 저장소로 해석할 수 있다.

## 7.8 SQL DDL Applicability

SQL DDL은 사용하지 않는다. vector document와 metadata는 ChromaDB client API로 생성/갱신/삭제되며, VaultState는 JSON atomic write로 관리된다. 향후 SQLite 또는 PostgreSQL로 이전할 경우 chunk\_id, source, relative\_path, page, content\_hash, synced\_at을 중심 키로 하는 relational schema를 별도 정의해야 한다.

# 8 Testing Plan

## 8.1 Objectives

테스트의 목표는 설정 검증, 동기화 안정성, RAG 응답 흐름, UI 주요 기능, 실패 상황 처리가 의도대로 동작하는지 확인하는 것이다.

## 8.2 Testing Policy

테스트 정책은 설정 검증, 저장소 안정성, 검색 품질, 스트리밍 응답, UI 사용자 흐름을 계층별로 검증하는 것이다. 자동화 테스트는 단위와 통합 경계를 우선하고, 릴리스 전에는 실제 Vault 샘플을 이용한 수동 시나리오를 병행한다.

### 8.2.1 Development Testing

현재 tests/test\_config.py는 VaultSection의 빈 path 처리와 MCPServerSpec의 stdio/http 변환을 검증한다. 개발 단계에서는 이 범위를 확장해 다음 항목을 테스트해야 한다. Vault scanner가 제외 디렉토리와 확장자 필터를 정확히 적용하는지 확인한다. VaultManager.compute\_delta가 add, update, delete, touched-but-unchanged 파일을 정확히 분류하는지 확인한다. chunk\_pages가 page/location metadata를 유지하는지 확인한다. RAG graph가 검색 결과 없음, rewrite 횟수 제한, MCP route 조건을 올바르게 처리하는지 확인한다.

### 8.2.2 Release Testing

릴리스 전에는 backend pytest, frontend npm run build, 대표 Vault 샘플을 이용한 수동 sync, Chat streaming smoke test를 수행한다. OPENAI\_API\_KEY가 없는 상태, 빈 Vault 상태, 잘못된 Vault path, ChromaDB clear 후 재동기화 상태도 확인해야 한다.

### 8.2.3 User Testing

사용자 테스트는 다음 흐름을 기준으로 한다. 첫째, 사용자가 /onboarding에서 Vault path를 입력하고 첫 동기화가 완료되는지 확인한다. 둘째, Vault 화면에서 파일 목록과 마지막 동기화 시간이 보이는지 확인한다. 셋째, Chat 화면에서 문서 기반 질문을 입력했을 때 출처 badge가 표시되는지 확인한다. 넷째, 일반 인사 입력이 RAG 없이 일반 응답으로 처리되는지 확인한다. 다섯째, Settings 화면에서 retrieval 값을 변경한 뒤 다음 질문부터 반영되는지 확인한다.

### 8.2.4 Testing Cases

주요 테스트 케이스의 ID, 테스트 내용, 기대 결과는 아래 표에 정리했다.

**Figure 8.1 Testing Cases**

| TC ID | 테스트 내용                                | 기대 결과                             |
|-------|---------------------------------------|-----------------------------------|
| TC-01 | 빈 OPENAI_API_KEY 상태에서 Chat 요청         | API key 안내 메시지 반환                 |
| TC-02 | Vault path 미설정 상태에서 sync 요청           | HTTP 400 반환                       |
| TC-03 | 지원하지 않는 확장자 파일이 Vault에 존재             | scan_vault 결과에 포함되지 않음            |
| TC-04 | max_file_mb 초과 파일 sync                | skipped 목록에 reason: too_large로 반환 |
| TC-05 | PDF 파일 로드                             | 페이지 단위 LoadedPage로 변환             |
| TC-06 | DOCX 파일 로드                            | paragraph와 table text가 모두 로드됨     |
| TC-07 | 인덱싱된 파일 삭제 후 sync                     | ChromaDB에서 해당 source 청크 삭제        |
| TC-08 | POST /api/chat/stream 호출              | meta 이후 token 또는 done 반환          |
| TC-09 | force=false로 POST /api/index/clear 호출 | HTTP 400 반환                       |
| TC-10 | SettingsPatch에 알 수 없는 key 포함          | HTTP 400 반환                       |

## 8.3 테스트 범위 보완

Unit test는 설정 로더뿐 아니라 ChromaStore.search의 빈 쿼리 처리와 score 정렬, OpenAILLM.grade의 yes/no 판정 및 score fallback, \_coerce\_history의 dict/tuple/ChatMessage 정규화, MCPClient 결과 정규화를 포함해야 한다.

Integration test는 Vault 동기화 E2E, 델타 동기화, RAG 그래프, ask\_stream 이벤트 순서를 검증한다. 대표 시나리오는 PDF 추가 후 ChromaDB count 증가, 파일 수정 후 updated=1, meta -> token\* -> done 순서, MCP 실패 시 내부 문서 답변 유지이다.

Performance와 reliability 검증은 로컬 단일 사용자 환경에 맞춘다. 10페이지 PDF 인덱싱 30초 이내, 1000개 청크 검색 1초 이내, 첫 토큰 5초 이내를 목표로 두고, RateLimitError 재시도, Vault 없음, 빈 Vault, API key 없음, OCR 미지원 PDF 처리도 회귀 항목으로 관리한다.

# 9 Development Plan

## 9.1 Objectives

이 장은 시스템 구현과 실행을 위한 개발 환경, 도구, 제약 조건, 의존성을 설명한다.

## 9.2 Backend Environment

Backend는 Python 3.11 이상을 요구한다. 패키지는 pyproject.toml에 정의되어 있으며 pydantic, pydantic-settings, pyyaml, python-dotenv, openai, langchain, langchain-openai, langgraph, langchain-chroma, chromadb, pypdf, python-docx, fastapi, uvicorn 등이 포함된다. CLI entry point는 oh-my-neuro = app.main:main이다.

## 9.3 Frontend Environment

Frontend는 Vite, React 19, TypeScript, React Router, Tailwind CSS, lucide-react, react-markdown, remark-gfm을 사용한다. 주요 명령은 npm ci --prefix frontend, npm run build --prefix frontend, npm run dev --prefix frontend이다. 프로덕션 UI는 frontend/dist가 존재할 때 FastAPI가 정적 파일로 서빙한다.

## 9.4 Configuration Environment

OPENAI\_API\_KEY는 .env 또는 환경 변수로 설정한다. 기본 UI host는 127.0.0.1이고 port는 7860이다. MCP는 configs/app.yaml에서 기본적으로 false로 설정되어 있으며, 활성화 시 configs/mcp\_servers.yaml의 korean\_law 서버 설정을 사용한다.

## 9.5 Constraints

시스템은 로컬 실행을 전제로 한다. 원본 문서는 서버에 업로드되지 않고 지정된 Vault 디렉토리에서 읽힌다. 임베딩 및 답변 생성은 OpenAI API에 의존하므로 네트워크와 API key가 필요하다. ChromaDB collection은 로컬 data/chroma에 저장되므로 사용자의 파일 시스템 권한과 디스크 상태에 영향을 받는다. 브라우저 세션 기록은 localStorage에만 저장되므로 브라우저 또는 프로필이 바뀌면 공유되지 않는다.

## 9.6 Assumptions and Dependencies

사용자는 Python 3.11, Node.js/npm, OpenAI API key를 준비할 수 있다고 가정한다. Vault 디렉토리에는 지원 확장자의 텍스트 추출 가능한 파일이 들어 있다고 가정한다. PDF 스캔 이미지처럼 텍스트 추출이 불가능한 파일은 현재 OCR 대상이 아니다. MCP 기능은 외부 MCP 서버 실행 가능성과 도구 응답 형식에 의존한다.

## 9.7 Implementation Roadmap

단기 개선 항목은 scanner, delta, ingestion, RAG graph 단위 테스트 확대이다. 중기 개선 항목은 OCR 지원, 사용자 설정 영속화, UI 오류 상태 개선, MCP status 노출이다. 장기 개선 항목은 사용자별 profile, 문서별 권한, 배포 환경 분리, observability 추가이다.

## 9.8 개발 품질 기준

개발자는 Python 의존성 설치, frontend build, backend pytest, 대표 Vault 수동 sync, Chat streaming smoke test를 릴리스 전 최소 확인 항목으로 둔다. 프로덕션 UI는 frontend/dist가 있어야 FastAPI에서 정적 파일로 서빙된다.

로컬 실행 제약은 설계의 핵심 전제이다. 원본 문서는 Vault에 남아 있고, 서버에는 청크와 메타데이터만 저장된다. OpenAI API 호출에는 유효한 API key와 네트워크가 필요하며, ChromaDB 성능은 로컬 디스크와 인덱스 규모에 영향을 받는다.



## 10 Supporting Information

### 10.1. Software Design Specification

소프트웨어 요구사항 명세서 IEEE 권장사항 (IEEE Recommend Practice for Software Requirements Specifications, IEEE-Std-830)에 따라 작성되었다.

### 10.2. Document History

| Date     | Description                                        | Version | Writer |
|----------|----------------------------------------------------|---------|--------|
| 26.05.31 | Overall Design Lead and Requirements Specification | 1.0     | 배석현    |
| 26.05.31 | System Architecture - Overall and Frontend         | 1.0     | 유원석    |
| 26.05.31 | System Architecture - Backend and Protocol Design  | 1.0     | 이윤지    |
| 26.05.31 | Database Design and Testing Plan                   | 1.0     | 이준     |
| 26.05.31 | Development Plan and Supporting Information        | 1.0     | 신현호    |